Database Design — Samples (Revised): Tables and ER Diagrams

About this edition

This is the corrected companion to the revised lesson. It is the "picture book": every idea is shown as a **real table with sample rows** and as a **crow's-foot diagram**. It faithfully follows the uploaded samples file (which I could read in full) and folds in the Task-2 corrections. Anything I changed or added is marked **[correction]** so you can see exactly what moved.

Corrections applied here: **C1** a note that INTEGER ids and UUIDs are two kinds of the same surrogate-key idea; **C2** the prescription ternary now decides whether its natural triple is UNIQUE; **C3** clear "pick the pair-key vs a surrogate-id bridge when…" guidance; **C4** every good original addition kept (frozen fact, reserved words, crow's-foot mirroring); **C5** the two ten-step methods replaced by one pointer to the merged routine in the revised lesson.

The sample rows here are the **same rows** used in the revised lesson, so the two files tell one connected story: the same members, books, copies, loans, groups, students, orders and products appear in both.

How to read the ER diagrams (crow's-foot in one box)

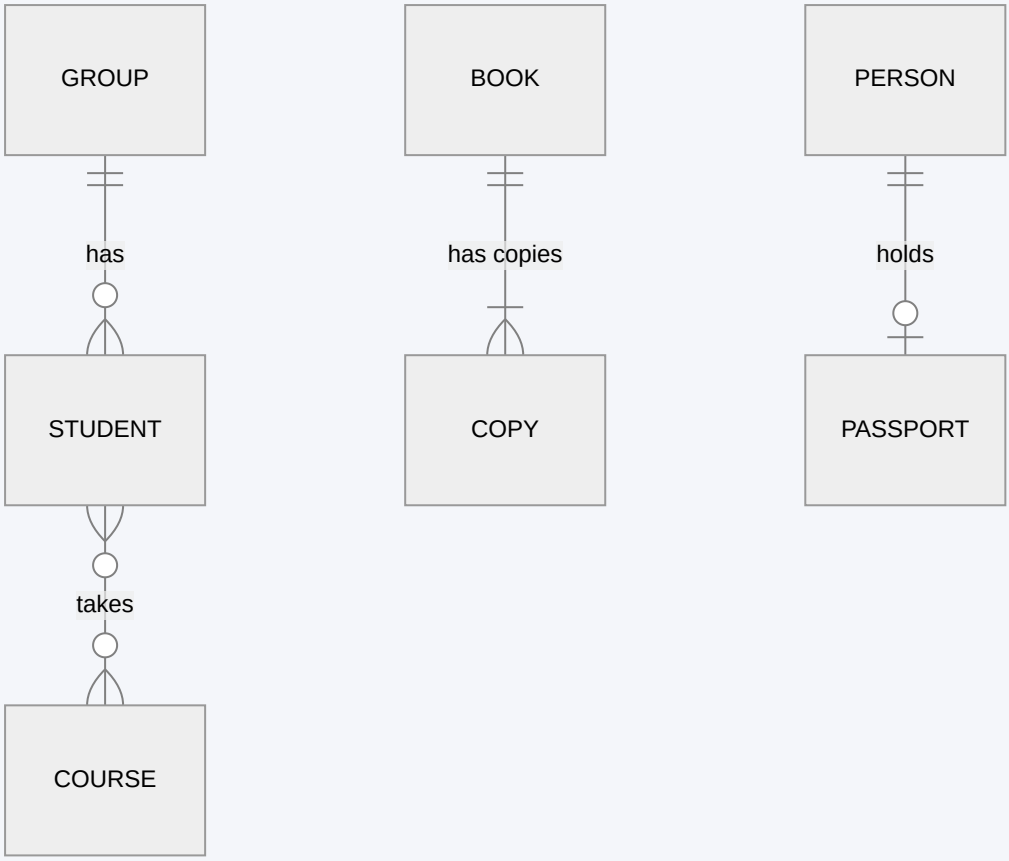
Each line between two boxes has a symbol at **each end**. Read the symbol at the *far* end to learn "how many of that thing."

Symbol	Means
`||`	exactly one
`o|`	zero or one (optional)
`|{`	one or many
`o{`	zero or many (optional, many)

In Mermaid the two ends are written together, e.g. `||--o{`:

- `A ||--o{ B` → one A has zero-or-many B; each B has exactly one A.
- `A ||--|{ B` → one A has one-or-many B; each B has exactly one A.
- `A }o--o{ B` → many-to-many (both optional).
- `A ||--|| B` → one-to-one.

A tiny gallery you can read back as sentences:



Line	Sentence
`GROUP ||--o{ STUDENT`	one group has zero-or-many students; each student has exactly one group
`BOOK ||--|{ COPY`	one book has one-or-many copies; each copy is of exactly one book
`PERSON ||--o| PASSPORT`	one person holds zero-or-one passport; each passport is of one person
`STUDENT }o--o{ COURSE`	many-to-many — resolve it with a bridge (Lesson 7)

[correction] Mirroring note. The symbols are *mirrored* on the two ends. `o{` on the right end and `}o` on the left end mean the same thing (zero-or-many); you read whichever one is nearest the box you are asking about.

A note on id style (read once)

[correction C1] Every sample below uses small INTEGER ids (1, 2, 3) because they are easy to read while learning. student-panel instead uses a **UUID** (a long random id) as the primary key, plus a short unique human code (e.g. `student_code`). These are **the same idea** — a surrogate key the system invents — in two costumes. When you see `member_id = 1` here and a long random id in student-panel, that is not a conflict; it is the same decision made two ways.

The same `student` row in both costumes, so you can see they are the same design:

costume	id column	sample value	human code kept UNIQUE
INT (here)	student_id	1	STU-0001
UUID (student-panel)	id	7f3a1c2e-9b55-4a0e-8d21-0c2b9c2b7a10	STU-0001

A note on reserved words

[correction C4, kept] Some words are reserved in SQL — `order`, `group`, `user` are the common traps. Name tables safely (`orders`, `groups`, `app_user`) so the database does not mis-read your table name as a command. student-panel (Postgres) does exactly this.

Reserved word	Safe table name
`order`	`orders`
`group`	`groups`
`user`	`app_user`

That is why the many-to-many lesson below uses `orders`, not `order`.

Lesson 1 — Entity vs attribute

An **entity** is a kind of thing worth its own table. An **attribute** is a property of it (a column). Test: does it have its *own* facts that others point at? → entity. Is it just a value *of* something? → attribute.

`member` table (sample rows):

member_id	full_name	email	joined_on
1	Ada Lovelace	ada@example.com	2024-01-10
2	Alan Turing	alan@example.com	2024-02-02
3	Grace Hopper	grace@example.com	2024-02-15

Here `full_name`, `email`, `joined_on` are **attributes** of the **entity** `member`. "Ada Lovelace" is not a table — it is a value in one row.

Sorting the same nouns into entity vs attribute:

Thing in the world	Entity or attribute?	Why
member	**entity**	its own facts; many loans point at it
full_name	**attribute**	a single value belonging to one member
email	**attribute**	one value per member (kept UNIQUE)
group (that a member belongs to)	**entity**	shared by many members; has its own name



Lesson 2 — Keys: candidate, primary, natural vs surrogate

A **candidate key** uniquely identifies a row. The one you choose is the **primary key**. A **natural key** is real-world data (an ISBN); a **surrogate key** is a system-made id with no meaning.

`book` table — `isbn` is a natural candidate key, `book_id` is the surrogate PK:

book_id (PK)	title	author	isbn
1	The Pragmatic Programmer	Hunt & Thomas	9780135957059
2	Clean Code	R. C. Martin	9780132350884
3	Structure and Interpretation	Abelson & Sussman	9780262510875

The keys sorted out:

Key	Which column(s)	Role
candidate	`book_id` *and* `isbn`	each could identify a row on its own
primary	`book_id`	the one we chose — never changes, never repeats
natural	`isbn`	real-world id; kept as `UNIQUE` so you can still look up by it
surrogate	`book_id`	invented by the system, no business meaning

[correction C1] `book_id` is shown as INT here; student-panel would use a UUID. Both are surrogate keys. Keeping the natural key too (as `UNIQUE isbn`) lets you still look a book up by its real-world identifier.

Lesson 3 — Translating words into a model

A quick habit for turning a sentence into a design:

In the sentence…	Becomes…
a **noun** you keep facts about	an **entity** (table)
a **property** of that noun	an **attribute** (column)
a **verb** linking two nouns	a **relationship**

Example: "A **member** *borrows* a **copy** of a **book**; each loan has a **due date**." → entities `member`, `copy`, `book`; relationship *borrows* (a `loan`); attribute `due_on`.

The same sentence, parsed word by word:

Word in the sentence	Part of speech	Becomes
member	noun (we keep facts)	entity `member`
borrows	verb	relationship → the `loan` table
copy	noun (we keep facts)	entity `copy`
book	noun (we keep facts)	entity `book`
due date	property of the loan	attribute `due_on`

Lesson 4 — Relationships and cardinality

The three shapes, each shown with sample rows and a diagram.

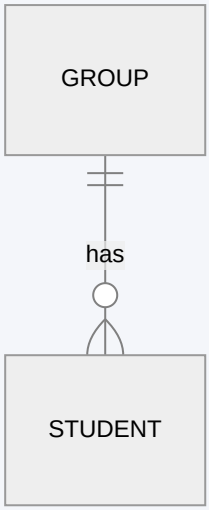
1:N — one group, many students (FK on the many side):

`group` table:

group_id	name
1	Morning
2	Evening

`student` table (FK `group_id` on the many side):

student_id	full_name	group_id
1	Ada	1
2	Alan	1
3	Grace	2



"One group has zero-or-many students; each student has exactly one group." See how students 1 and 2 both carry `group_id = 1` — that is "many students, one group" on the page.

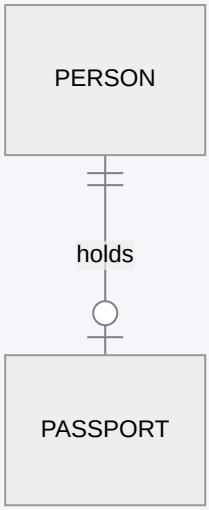
1:1 — one person, one passport. FK goes on the side that is never empty, with `UNIQUE` + `NOT NULL`:

`person`:

person_id	full_name
1	Ada
2	Alan

`passport` (FK `person_id`, UNIQUE so no person has two):

passport_id	person_id (UNIQUE)	number
1	1	X1234567



"One person holds zero-or-one passport; each passport belongs to exactly one person." The `UNIQUE` on `person_id` is what blocks a second passport for Ada — without it, this would be a 1:N.

M:N — students and courses. Neither side can hold the FK, so a **table in the middle** carries one FK to each — see Lesson 7.

Lesson 5 — Referential integrity (no orphans)

Every foreign key must point at a row that exists. If `loan.member_id = 9` but there is no member 9, that loan is an **orphan**, and the database refuses it.

`loan` with a bad FK (rejected):

loan_id	member_id	copy_id
1	1	1
2	**9**	2

The rule is enforced by the engine, not by hope: declare `member_id` as a foreign key `REFERENCES member(member_id)` and the insert of loan 2 fails.

The reverse protection is the delete rule. Because loan 1 points at member 1, the database will not silently let member 1 vanish — it either blocks the delete (RESTRICT), removes the loans too (CASCADE), or blanks the pointer (SET NULL). Those three choices are Lesson 17.

If you try to delete member 1 …	RESTRICT	CASCADE	SET NULL
… while loan 1 still points at them	blocked	loan 1 deleted too	loan 1 kept, `member_id` → NULL

Lesson 6 — Foreign-key placement

The FK lives on the **many** side of a 1:N, and on the **never-empty** side of a 1:1. A quick check: "which row can there be *many* of for one of the other?" — that row carries the pointer. One group → many students, so `group_id` sits on `student` (Lesson 4), never the reverse.

Worked both ways, to show why the reverse fails:

Put `group_id` on …	Result
`student` (correct)	many students share one `group_id` → "one group, many students" ✓
`group` (wrong)	one group could name only one student → breaks the "many" ✗

The same check across our examples:

Relationship	"Many of which for one of the other?"	FK goes on
group ↔ student	many students per group	`student.group_id`
book ↔ copy	many copies per book	`copy.book_id`
member ↔ loan	many loans per member	`loan.member_id`
person ↔ passport (1:1)	never-empty side is passport	`passport.person_id` UNIQUE

Lesson 7 — The M:N bridge: orders and products

An order contains many products; a product appears on many orders. That M:N is resolved by a bridge, `order_line`. (Note the table is `orders`, not `order` — reserved word.)

`orders`:

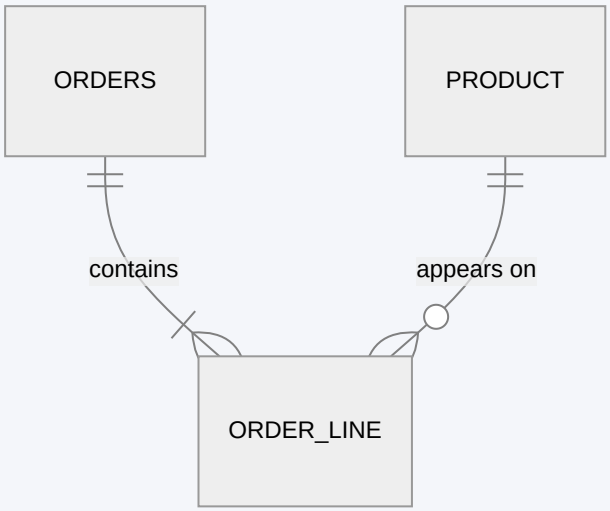
order_id	ordered_on	customer_id
1	2024-03-01	1
2	2024-03-04	2

`product`:

product_id	name	list_price
1	Blue pen	1.50
2	Red pen	1.50

`order_line` (the bridge — one row per product on an order):

order_id	product_id	quantity	unit_price_at_sale
1	1	3	1.50
1	2	2	1.20
2	1	1	1.50



Reading it: one order contains one-or-many lines (an order with no lines is not really an order → `|{`); one product appears on zero-or-many lines. The pair `(order_id, product_id)` is the key, so a product can appear at most once per order — order 1 lists the blue pen once, with a quantity of 3, not three rows.

[correction C4, kept] Frozen fact. `unit_price_at_sale` is stored on purpose — it is the price paid *at that moment*. Even if `product.list_price` changes tomorrow, the old order must still show 1.20. This is *not* a "derive, don't store" violation; it is a captured historical fact.

Look at the rows: the red pen's list price is 1.50, but order 1 paid 1.20 (a one-off discount). If the line did not freeze its own price, that discount would be lost the moment anyone looked at the order again.

[correction C3] Which key for the bridge?

- Use the **pair as the key** — `PRIMARY KEY(order_id, product_id)` — when the pairing is naturally unique (a product appears at most once per order) and nothing else needs to point *at* a line.
- Give the bridge **its own surrogate id** (`order_line_id`) **when** something else must reference a single line (e.g. a returns table), **and** keep `UNIQUE(order_id, product_id)` so you still cannot list the same product twice on one order. Decide this on purpose, not by habit.

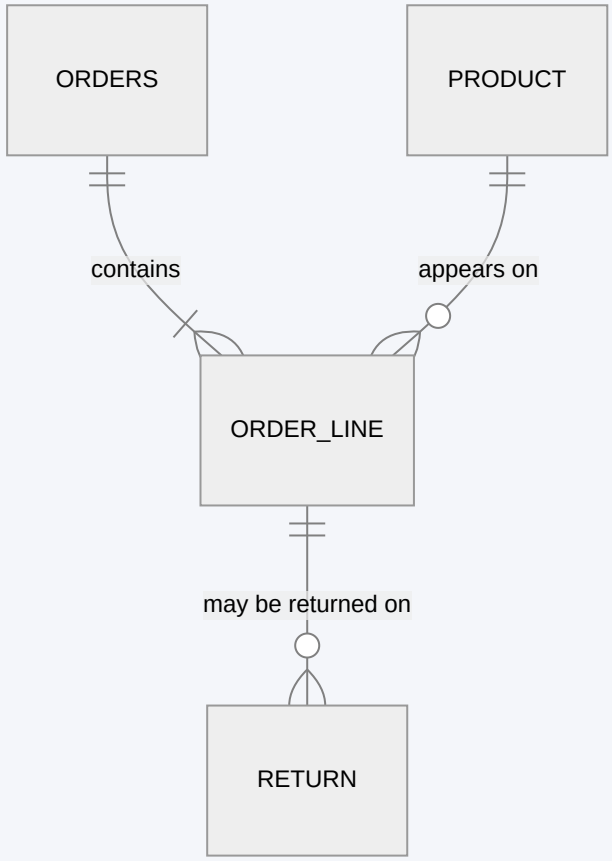
The surrogate-id variant, when a `return` must point at one specific line:

`order_line` (now with its own id, `UNIQUE(order_id, product_id)` still enforced):

order_line_id	order_id	product_id	quantity	unit_price_at_sale
100	1	1	3	1.50
101	1	2	2	1.20

`return` (points at a single line):

return_id	order_line_id	returned_qty	reason
1	101	1	faulty



Choose	When
pair key `(order_id, product_id)`	the pairing is unique and nothing points at a line
surrogate `order_line_id` + `UNIQUE(order_id, product_id)`	something else (a return) references one line

Lesson 8 — Enrolment and course offerings (two valid models)

Students enrol in courses, but a course runs in different semesters. There are two correct ways to model "the running of a course," and the samples show both.

Model A — a `teaching` bridge keyed by the natural pair `(course, semester)`:

`teaching`:

course_id	semester	teacher_id
1	2024-Fall	7
1	2025-Spr.	9

Key = `(course_id, semester)`.

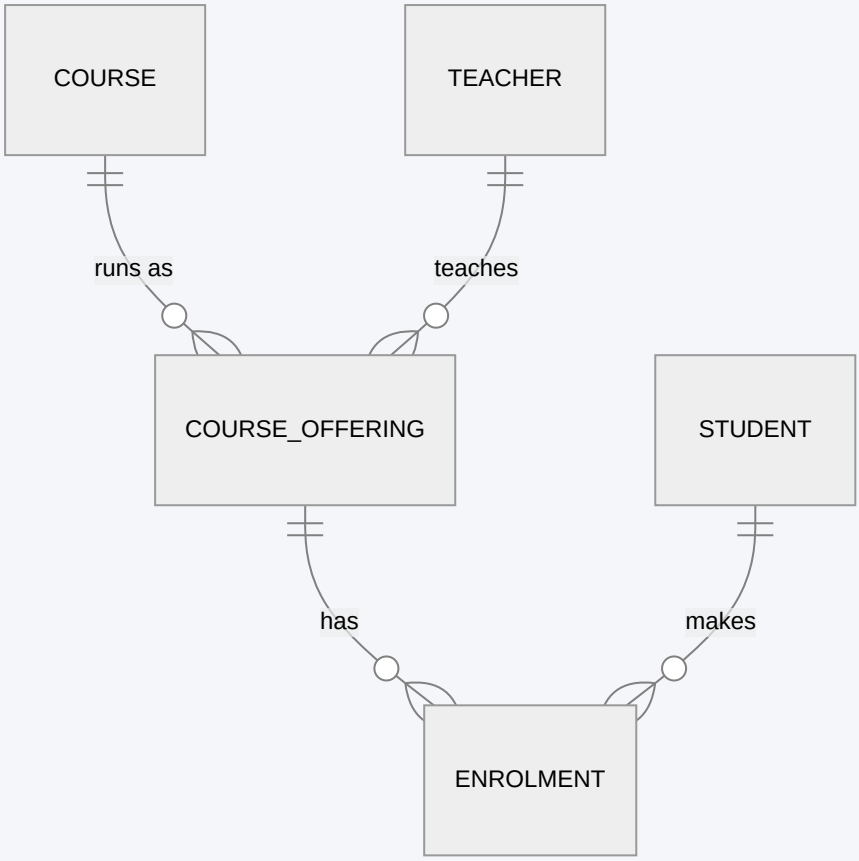
Model B — a `course_offering` with its own surrogate id (so enrolments and timetables can point at *one* offering):

`course_offering`:

offering_id	course_id	semester	teacher_id
1	1	2024-Fall	7
2	1	2025-Spr.	9

`enrolment` then references `offering_id`:

offering_id	student_id	grade
1	1	A
1	2	B



[correction C4] Pick-when. Choose **Model A** (the pair-key) when the pairing is naturally unique and nothing else points at it. Choose **Model B** (surrogate id) when other tables must reference one offering — then keep `UNIQUE(course_id, semester)` on it. Same rule as the order_line bridge.

The decision in one line each — this is the "pick this model when…" guide:

Pick	When	Must also add
Model A — pair key `(course_id, semester)`	nothing else points at a single running of the course	nothing extra
Model B — surrogate `offering_id`	enrolments / timetables must reference one offering	`UNIQUE(course_id, semester)`

Notice Model B is just the order_line surrogate choice again (Lesson 7): the same rule governs *every* bridge you ever build, so once you can make this call you can make it everywhere.

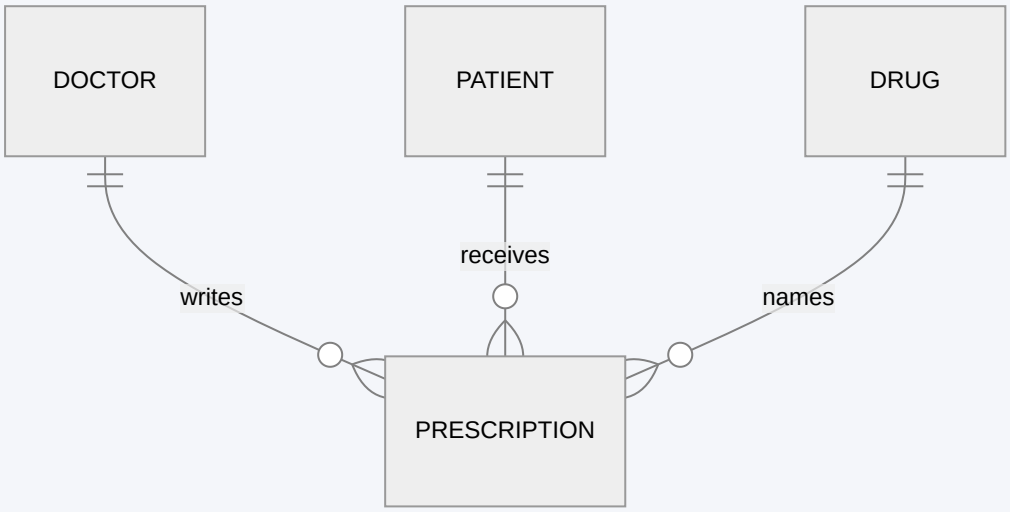
Lesson 9 — Ternary relationships: prescriptions

A fact that genuinely needs **three** things at once. "Doctor D prescribes drug G to patient P." Dropping any one noun destroys the fact — that is the **disguise test** (if the sentence is

still true after you remove a noun, it was not really a ternary).

`prescription`:

prescription_id	doctor_id	patient_id	drug_id	prescribed_on
1	7	1	3	2024-03-01
2	7	1	3	2024-06-01



The disguise test applied, drop each noun in turn:

Drop …	What's left	Still a complete fact?
the doctor	"drug G given to patient P"	no — *who ordered it?*
the patient	"doctor D prescribes drug G"	no — *to whom?*
the drug	"doctor D treats patient P"	no — *with what?*

All three are load-bearing, so it is a true ternary — a middle table with three FKs.

[correction C2] Decide the UNIQUE on purpose. With its own `prescription_id`, ask: should the natural triple `(doctor_id, patient_id, drug_id)` be UNIQUE? Here **no** — the same doctor may prescribe the same drug to the same patient again later (rows 1 and 2 differ only by date), so the date is part of what makes each prescription distinct. If repeats were *not* allowed, you would add `UNIQUE(doctor_id, patient_id, drug_id)`. Either way, make the choice consciously — exactly as you did for `order_line`.

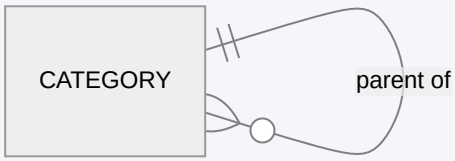
Does the real world allow a repeat triple?	Then …
yes (same doctor, drug, patient on a later date)	**no** triple-UNIQUE; the date distinguishes rows (our case)
no (each combination may occur once)	add `UNIQUE(doctor_id, patient_id, drug_id)`

Lesson 10 — Self-reference: a category tree

A category can sit inside another category. One table points at itself.

`category`:

category_id	name	parent_id
1	Books	*(null)*
2	Fiction	1
3	Science	1
4	Physics	3



`parent_id IS NULL` marks a **root** (Books). Never use `parent, grandparent, great_grandparent` columns — that caps the depth and repeats the pattern. student-panel uses exactly this self-reference shape for its group and parameter trees.

Reading the rows as a tree, each row names only its *immediate* parent:

```
Books (1, root)
├─ Fiction (2 → 1)
└─ Science (3 → 1)
   └─ Physics (4 → 3)
```

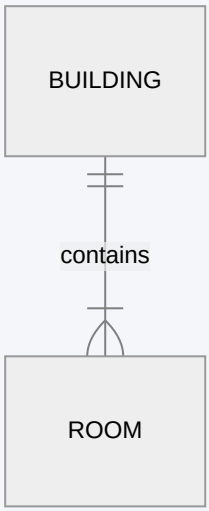
Add a fifth level ("Quantum Physics" under Physics) and you insert one row with `parent_id = 4` — no schema change, ever. That is the whole point of the single `parent_id` column.

Lesson 11 — Weak (dependent) entity

A weak entity cannot be identified on its own; it only exists as part of a parent. An `order_line` is weak — "line 2" means nothing without its order. Another classic: a building's `room`.

`room` (identified by its building **plus** its own number):

building_id	room_no	capacity
1	101	30
1	102	25
2	101	40



Key = `(building_id, room_no)`. Room 101 exists twice — once per building — and that is correct, because the building is part of its identity.

For contrast, the `building` parent that gives those rooms their identity:

building_id	name
1	Science Hall
2	Arts Hall

So "room 101" is ambiguous, but "room 101 in building 1 (Science Hall)" is exact. The delete rule here is almost always CASCADE: demolish the building (delete it) and its rooms go with it — a room has no meaning once its building is gone.

Lesson 12 — The five hard patterns, and "the one move"

Pattern	The fix
M:N (many-to-many)	a table in the middle (a bridge)
a link with its own facts	that link becomes a table
a three-way fact (ternary)	a table joining all three
1:1	usually merge into one table, unless a reason to split
self-reference (tree)	one table with a `parent_id` FK to itself

The one move: whenever a relationship is M:N, *has its own facts*, or joins three or more things, it becomes **a table in the middle**. Learn that one move and most "hard" designs become routine.

Each pattern tied to an example you have already seen in this file:

Pattern	Seen as	The middle/extra table
M:N	students ↔ courses (Lesson 4, 7)	`enrolment` / `order_line`
link with its own facts	a loan's dates, an enrolment's grade	`loan` / `enrolment`
ternary	doctor × patient × drug (Lesson 9)	`prescription`
1:1	person ↔ passport (Lesson 4)	usually merged; else FK on `passport`
self-reference	category tree (Lesson 10)	`category.parent_id`

Lesson 13 — First normal form (1NF): one value per cell

Broken — a list stuffed into one cell:

member_id	full_name	phones
1	Ada	555-1111, 555-2222

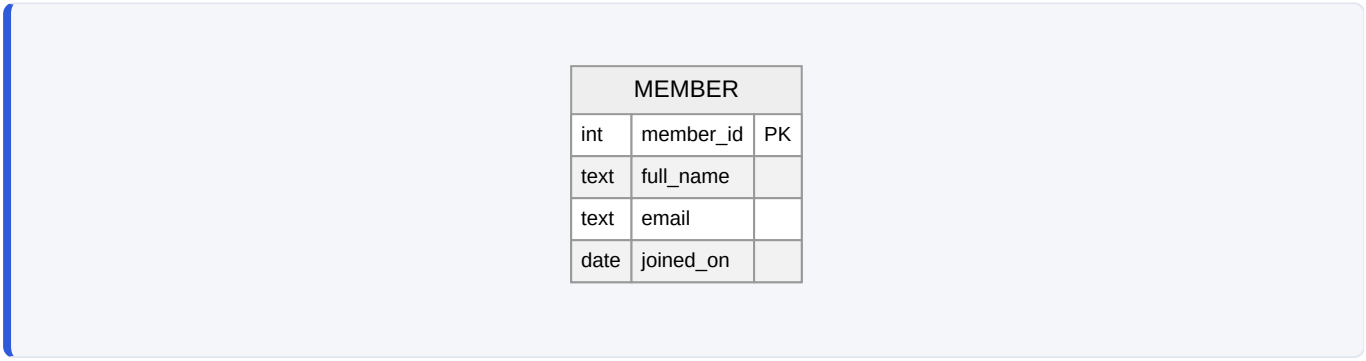
You cannot ask "who has 555-2222?" cleanly. **Fix** — one phone per row, in its own table:

`member`:

member_id	full_name
1	Ada

`phone`:

phone_id	member_id	number
1	1	555-1111
2	1	555-2222



0

Now "who has 555-2222?" is a single lookup in `phone`, and counting a member's phones is just counting rows. The broken version also silently decides how many phones fit; the fixed version has no limit.

Lesson 14 — Second normal form (2NF): the whole key

2NF matters when the key is **composite**. A non-key column must depend on the *whole* key, not part of it.

Broken — `order_line` keyed by `(order_id, product_id)` but carrying `product_name` (which depends on `product_id` alone):

order_id	product_id	product_name	quantity
1	1	Blue pen	3
2	1	Blue pen	1

Fix — move `product_name` to `product`, where it depends on the full key `product_id`. `order_line` keeps only `quantity` (and the frozen `unit_price_at_sale`).

`product`:

product_id	name	list_price
1	Blue pen	1.50

`order_line`:

order_id	product_id	quantity	unit_price_at_sale
1	1	3	1.50
2	1	1	1.50

The repeated "Blue pen" was the warning sign: rename the product and the broken table would need fixing on every line. In the fixed shape the name lives in one place.

Lesson 15 — Third normal form (3NF): nothing but the key

No non-key column may depend on another non-key column.

Broken — `student` carrying its group's name:

student_id	full_name	group_id	group_name
1	Ada	1	Morning
2	Alan	1	Morning

`group_name` depends on `group_id`, not on `student_id`. **Fix** — keep `group_name` only in `group`; `student` holds just the FK `group_id`.

`group`:

group_id	name
1	Morning
2	Evening

`student`:

student_id	full_name	group_id
1	Ada	1
2	Alan	1
3	Grace	2

Rename group 1 to "AM" in the broken table and you must edit two student rows (and miss one at your peril); in the fixed shape you edit a single row of `group`.

The mnemonic for all three: **"the key, the whole key, and nothing but the key."**

Form	Catch	Means
1NF	the key	one value per cell, no repeating columns
2NF	the whole key	no dependency on part of a composite key
3NF	nothing but the key	no dependency on another non-key column

[correction C5] The design routine. The samples file and the primer each had their own ten-step method. They are the same method sliced differently; this edition replaces both with **one merged, numbered routine** — see **Part 9 of the revised lesson** (`database-design-revised-lesson.md`). Use that single list.

Lesson 16 — Derive, don't store (and the one exception)

A value you can recompute from others should **not** be stored — storing it lets it drift out of step. But a value captured *at a moment in time* is a **frozen fact** and must be kept.

Value	Store it?	Why
an order's total	**No** — derive	it is just the sum of its lines
a student's age	**No** — derive	compute from `born_on` and today
`unit_price_at_sale`	**Yes** — store	the price paid then; must not change
an invoice's issued amount	**Yes** — store	a legal record of that moment

The test: *can I always recompute it from data that will not change?* If yes, derive. If the source can change but this value must not, freeze it.

Seen in rows — the order total is *not* a column; it is read off the lines:

`order_line` for order 1:

order_id	product_id	quantity	unit_price_at_sale	(line value)
1	1	3	1.50	4.50
1	2	2	1.20	2.40

Order 1's total is **6.90**, computed on demand (3×1.50 + 2×1.20). Store it and the day someone edits a line, the stored total lies. The `unit_price_at_sale` values, by contrast, are frozen on each line precisely so that computation stays true to what was actually paid.

Lesson 17 — ON DELETE rules (say the rule as a sentence)

When a parent row is deleted, what happens to its children?

Rule	Say it aloud	Use when
`RESTRICT`	"you may not delete a member who still has loans"	the child must not be orphaned or silently lost
`CASCADE`	"deleting an order deletes its order lines too"	the children have no life without the parent
`SET NULL`	"deleting a teacher leaves the course with no teacher"	the link is optional and may simply become empty

[correction C4, kept] Always read your chosen rule as a plain sentence before accepting it — the sentence tells you instantly whether it is the behaviour you want. student-panel leans on `RESTRICT` + soft-delete (an `archived_at` flag) rather than hard `CASCADE`, so history is never truly lost.

The three rules against the *same* parent delete, so the difference is concrete — delete order 1, which has two `order_line` children:

Rule on `order_line.order_id`	What happens to order 1's lines
RESTRICT	the delete is blocked while the two lines exist
CASCADE	both lines are deleted along with the order
SET NULL	(needs a nullable FK) the lines survive but point at no order — rarely what you want for a weak child

For a weak child like `order_line`, CASCADE is the honest choice; for history you want to keep, RESTRICT + soft-delete wins. Match the rule to the sentence.

Lesson 18 — A full worked design: a small gym

Requirement: "Members join the gym. Trainers run classes. A class happens at a time and has a capacity. Members book a place in a class."

Entities: `member`, `trainer`, `gym_class`, and a booking bridge (M:N member↔ class, with its own fact `booked_on`).

Final tables:

```
member(member_id PK, full_name, email UNIQUE, joined_on)
trainer(trainer_id PK, full_name)
gym_class(class_id PK, title, starts_at, capacity, trainer_id FK -> trainer)
booking(member_id FK -> member, class_id FK -> gym_class, booked_on,
        PRIMARY KEY(member_id, class_id))
```



1

Reading it: one trainer runs zero-or-many classes; one class has zero-or-many bookings; one member makes zero-or-many bookings. The `(member_id, class_id)` key stops a member booking the same class twice. "Capacity" is a plain attribute; "places left" is **derived** (capacity minus booked rows), so it is *not* stored.

Now the same design filled with rows so you can answer real questions.

`member`:

member_id	full_name	email	joined_on
1	Ada Lovelace	ada@example.com	2024-01-10
2	Alan Turing	alan@example.com	2024-02-02
3	Grace Hopper	grace@example.com	2024-02-15

`trainer`:

trainer_id	full_name
7	Lin Zhao
8	Marco Rossi

`gym_class` (FK `trainer_id`):

class_id	title	starts_at	capacity	trainer_id
1	Spin	2024-04-10 18:00	12	7
2	Yoga	2024-04-11 09:00	10	8

`booking` (bridge; key `(member_id, class_id)`):

member_id	class_id	booked_on
1	1	2024-04-01
2	1	2024-04-02
3	2	2024-04-03

Questions answered against these rows:

- **Who is booked into Spin (class 1)?** `booking` rows with `class_id = 1` → members 1 and 2 → **Ada and Alan**.
- **How many places are left in Spin?** `capacity` 12 minus 2 booked rows → **10 left** (derived, never stored).
- **Which trainer runs Yoga (class 2)?** `gym_class.trainer_id = 8` → **Marco**.
- **Can Ada book Spin again?** No — the key `(1, 1)` already exists, so the duplicate is refused. That is the composite key doing its job.

Lesson 19 — Eight self-check tests

Cover the answers; say yours aloud first.

1. **Is `colour` an entity or an attribute?** → Depends on purpose: an attribute when it is just a value; an entity when colours have their own facts others share. *(The corner shop keeps no facts about "red", so it is a value; the paint factory records red's code, base and supplier, so it earns a table.)*

2. **Where does the FK go in a 1:N?** → On the **many** side. *(One group, many students → `group_id` on `student`.)*
3. **Where does the FK go in a 1:1?** → On the **never-empty** side, `UNIQUE` + `NOT NULL`. *(On `passport.person_id`, UNIQUE so no person gets two.)*
4. **How do you resolve an M:N?** → A table in the middle (a bridge), one FK to each side. *(students ↔ courses → `enrolment`.)*
5. **What does `parent_id IS NULL` mean in a tree?** → That row is a **root** (no parent). *(Books in the category tree of Lesson 10.)*
6. **Which normal form does a comma-separated list in a cell break?** → **1NF**. *(One value per cell; split the phones into their own table.)*
7. **Should you store an order's total?** → No — **derive** it. (But *do* store `unit_price_at_sale` — a frozen fact.) *(The total is the sum of lines, which are already stored; the sale price is captured against a price that will move.)*
8. **Read `MEMBER ||--o{ LOAN`.** → "One member has zero-or-many loans; each loan has exactly one member." *(The zero lets a brand-new member exist with no loans; the exactly-one forbids an orphan loan.)*

Glossary

See the full glossary in `database-design-revised-lesson.md`. The terms used most here: **entity** (a table/type), **attribute** (a column), **relationship** (a verb between entities), **cardinality** (how many on each side), **primary key** (the chosen unique id), **foreign key** (a pointer to another table's PK, on the many side), **surrogate key** (a system-made id — INT or UUID), **bridge** (the table in the middle of an M:N), **referential integrity** (no orphan FKs), **normalisation** (one fact in one place), **frozen fact** (a value captured at a moment in time and kept, not re-derived).

A one-line reminder for each, so this file stands on its own too:

Term	In one line
entity	a kind of thing worth its own table (a type)
attribute	one fact about an entity (a column)
relationship	a verb joining entities; a line in the diagram
cardinality	one/many on each side, stated both ways
participation	optional (min 0) or mandatory (min 1)
primary key	the chosen unique id for a row
foreign key	a pointer to another table's PK; lives on the many side
surrogate key	a system-made id with no meaning — INT or UUID
natural key	a real-world id (ISBN, email), kept UNIQUE
bridge	the middle table that resolves an M:N
referential integrity	every FK points at a row that exists
normalisation	one fact in one place (1NF/2NF/3NF)
frozen fact	a point-in-time value kept on purpose, not re-derived
self-reference	a `parent_id` pointing back at the same table (a tree)
weak entity	one that needs its parent to be identified

End of the revised samples companion. Read alongside `database-design-revised-lesson.md` (the full lesson), the hands-on `database-design-practice-lesson.txt`, and the concept bridge `database-design-companion-lesson.txt`.